

Travelling Salesman Problem (TSP)

Ahmed Al-khlil & Anas Al-abdolkarem & Osama mohamed

June 8, 2026

1 السيناريو

تخيل أن هناك شركة توصيل لوجستية تمتلك مركبة توصيل واحدة، وانطلاقاً من المقر الرئيسي للشركة، يتعين على السائق زيارة عدد محدد من المدن (أو العملاء) لتسليم البضائع، ثم العوده في نهايه المطاف الى نقطه البدايه (المقر الرئيسي). ولتحقيق اعلى مستوى من الكفاءة وتقليل التكاليف الاضافيه، يواجه السائق عده شروط صارمه:

1. يجب زياره كل مدينه مره واحده فقط.
2. يجب تقليل اجمالي المسافه المقطوعه (او اجمالي زمن الرحله او الوقود) الى الحد الادنى.
3. يجب العوده الى نقطه الانطلاق (المقر الرئيسي).
4. يمنع بتاتا اغلاق حلقه جزئيه ببعض المدن ثم العوده الى نقطه البدايه والخروج مره اخرى لاكمال باقي المدن.

1.1 التعقيد الحسابي للمشكلة

تكمن الصعوبه البالغه في هذه المشكله عندما يبدأ عدد المدن بالارتفاع، حيث يزداد عدد المسارات الممكنه بشكل "عاملي" ($n!$). لنستعرض كيف تتضخم الاحتمالات:

- اذا كان على السائق زياره 3 مدن فقط، فان حساب المسار الاقصر سهل للغاية لان الاحتمالات محدوده.
- اذا ارتفع العدد الى 10 مدن، يقفز عدد المسارات المحتمل الى اكثر من 180000 مسار بديل.
- اما اذا كان هناك 20 مدينه فقط، فان عدد الاحتمالات ينفجر ليصل الى رقم خيالي من الكوينتيليونات (10^{18} مسار).

بسبب هذا النمو الأسّي الهائل في الاحتمالات، يصبح من المستحيل على الحواسيب العاديه حساب واختبار كل مسار على حده للوصول الى الحل المثالي مع زياده حجم البيانات. لذلك، تصنف "مشكلة البائع المتجول" (TSP) كواحد من أشهر المسائل غير الحتميه متعدد الحدود الصعبه جداً (NP-hard)، وهي المسائل التي يسهل يصاغتها وفهمها، ولكن يصعب جداً حلها بشكل مطلق عندما تصبح واسعه النطاق.

2.1 انواع المشكله وعدد الوصلات

تنقسم الى نوعين رئيسيين بتاءاً على اتجاه الطرق:

1. **مشكله البائع الجوال المتناظره Symmetric TSP**: هنا تكون الطرق ذات اتجاهين ومتساويه في التكلفة. اي ان المسافه من المدينه A الى المدينه B, تساوي المسافه من A الى B. اي $(c_{ij} = c_{ji})$. ويحسب عدد الوصلات بالعلاقه:

$$NumberofEdges = \frac{n(n-1)}{2} ; \quad n \in N^*$$

2. **مشكله البائع الجوال غير المتناظره Asymmetric TSP**: وهنا تكون الطرق موجهة وتختلف تكلفها حسب الاتجاه $(c_{ij} \neq c_{ji})$. ويحسب عدد الوصلات بالعلاقه:

$$NumberofEdges = n(n-1) ; \quad n \in N^*$$

3.1 طرق حساب وتمثيل البيانات

عند حساب المسافات بين المدن وتجهيزها قبل ادخالها الى الخوارزميه كي لا تستهلك قدره المعالج في تكرار حسابات غير مهمه, ولحساب المسافات بين اي نقطتين هناك عده قوانين رياضيه, منها قانون حساب المسافه الاقليديه وهي المسافه المباشره بين نقطتين, وتعطى بالعلاقه التاليه:

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

ولتمثيل البيانات رياضيا او برمجياً يتم وضع البيانات في مصفوفه ثنائيه الابعاد 2D حجمها $n \times n$.

- السطر يعبر عن مدينه الانطلاق, والعمود يعبر عن مدينه الوصول.
- التقاطع بين السطر i والعمود يعبر عن قيمه التكلفة c_{ij} (المسافه او الزمن).
- القيم على القطر الرئيسي تكون دائماً صفراً او مالا نهائيه, لمنع السائق من الانتقال من المدينه الى المدينه نفسها.

4.1 اختيار الخوارزميه المناسبه

1.4.1 معايير المقارنه في اختيار الخوارزميه المناسبه

عند التعامل مع مشكله البائع المتجول TSP. لا يوجد مفهوم "الخوارزميه الافضل مطلقاً" دون النظر الى سياق البيانات وحجم المشكله. ان عمليه اختيار الخوارزميه المناسبه تحكمها

دائماً مقايضه اساسيه (Tread-off) بين ثلاثة عوامل رئيسيه وهي: الزمن المستغرق (Time), الموارد المتاحة (اي القدره الحسابيه والتخزينيه) (Calculation & Memory), والدقه المطلوبه (Accuracy).

بناءً على هذه الفلسفه, ولتحديد الخوارزميه التي تحقق التوازن الامثل بين كفاءه الاداء ودقه النتائج, قمنا باجراء مقارنة ممنهجه بين سته من ابرز الخوارزميات عالمياً, تتراوح بين الحلول الجشعه البسيطه, والبرمجه الديناميكيه المعقده, وصولاً الى الخوارزميات الهجينه المتطوره. وفقاً لخمسه معايير محوريه:

1. فئة الخوارزميه (Algorithm class): التصنيف البنيوي للخوارزميه.
2. فلسفه اتخاذ القرار (Descision logic): الاليه التي تتبعها الخوارزميه لاستكشاف المسارات.
3. التعقيد الحسابي والزماني (Time & Space complexity): مدى استهلاك الخوارزميه للوقت وموارد النظام مع نمو عدد المدن (n).
4. دقه الحل (Accuracy): مدى قرب مسار الناتج من الحل الاقصر المثالي.
5. مرونة البيانات (Data flexibility): القدره على التعامل مع القيود المختلفه والمسارات موجهة الاتجاه.

2.4.1 مقارنة الخوارزميات

• الجار لأقرب (Nearest Neighbor):

1. فئة الخوارزميه (Algorithm class): جشعه (Greedy Heuristic).
2. فلسفه اتخاذ القرار (Descision logic): اختيار المدينه الاقرب محلياً في كل خطوه دون النظر للمستقبل.
3. دقه الحل (Accuracy): سيئه الى متوسطه (غالباً اطول بحوالي 25% من الحل المثالي).
4. التعقيد الحسابي والزماني (Time & Space complexity): التعقيد الزماني اسي $O(n)$, والذاكره $O(n^2)$ أو $O(n)$.

• هليد-كارب (Held-Karp):

1. فئة الخوارزميه (Algorithm class): دقه فائقه (Exact/Dynamic Programming).
2. فلسفه اتخاذ القرار (Descision logic): تقسيم المشكله الى مشاكل فرعيه ابسط وتخزين حلولها لمنع تكرار العمليات الحسابيه وحساب الحل المثالي المطلق.
3. دقه الحل (Accuracy): الدقه مثاليه 100%, ولكن لاتستطيع معالجه اعداد مدن تزيد عن 32 مدينه تقريباً.
4. التعقيد الحسابي والزماني (Time & Space complexity): التعقيد الزماني انفجاري $O(2^n n)$, والتعقيد التخزيني $O(2^n n)$ هائل جداً.

• مستعمره النمل (Ant Colony Optimization):

1. فئة الخوارزميه (Algorithm class): ذكاء السراب (Metaheuristic).
2. فلسفه اتخاذ القرار (Descision logic): محاكاة سلوك النمل عبر افراز الفيرومون، والمسارات الاقصر تكتسب جذباً اقوى تدريجياً عبر ترسب الفيرومون.
3. دقه الحل (Accuracy): عاليه جداً (تقترب من المثاليه في الاحجام المتوسطه).
4. التعقيد الحسابي والزمني (Time & Space complexity): التعقيد الزمني $O(I \times A \times n^2)$ حيث I التكرارات و A عدد النمل. والتعقيد التخزيني $O(n^2)$.

• التطور الجيني (Generational Optimization):

1. فئة الخوارزميه (Algorithm class): جينيه تطوريه (Metaheuristic).
2. فلسفه اتخاذ القرار (Descision logic): توليد مجتمع من الحلول، ثم اجراء تزاوج (Crossover)، وطفرات (Mutation) للبقاء للاصلح والاقصر.
3. دقه الحل (Accuracy): متوسطه الى عاليه (تحتاج وقت طويل للتقارب).
4. التعقيد الحسابي والزمني (Time & Space complexity): التعقيد الزمني $O(I \times P \times n^2)$ حيث I التكرارات و P حجم المجتمع. والتعقيد التخزيني $O(nP)$.

• opt-2 & opt-3:

1. فئة الخوارزميه (Algorithm class): تحسين محلي (Local Search).
2. فلسفه اتخاذ القرار (Descision logic): قطع وتوصيل الوصلات بين المدن، حافتين في كل مره (2-opt)، او ثلاثه في كل مره (3-opt). وذلك لايجاد وصلات اقصر
3. دقه الحل (Accuracy): متوسطه الى جيده جداً
4. التعقيد الحسابي والزمني (Time & Space complexity): العقيد الزمني $O(n^2)$ for opt-2 و $O(n^3)$ for opt-3.

• LKH-3:

1. فئة الخوارزميه (Algorithm class): تحسين محلي متطور (Advanced Metaheuristic).
2. فلسفه اتخاذ القرار (Descision logic): تقطيع وتوصيل للوصلات بشكل ديناميكي متغير تابع ل K من الوصلات، (opt-k) مع ميزات ذكيه لمنع السقوط في الحلول المحليه الفرعيه وتدعم القيود المعقده.
3. دقه الحل (Accuracy): قريه جداً من المثاليه (99% الى 100%).
4. التعقيد الحسابي والزمني (Time & Space complexity): في التعقيد الزمني، تقترب جداً من $O(n^{2.2})$. وفي التعقيد التخزيني $O(n)$ ذو كفاءه عاليه جدا في اداره الذاكرة.

2 خوارزميه LKH-3

1.2 مدخل الى عبقرية خوارزميه LKH-3

في عام 1973، قدم العالمان Shen Lin و Brian Kernighan خوارزميه LK، والتي ابتكرت فكرة استخدام k-opt متغير. فبدلاً من تبديل عدد ثابت من الوصلات (مثل وصلتين

او ثلاث في كل مره), تغير الخوارزميه عدد تبديل الوصلات بناءً على المعطيات ديناميكياً للخروج من فخ الانماط دون المستوى الامثل (Suboptimal Patterns).

في عام 2000, احدث العالم Keld Helsgaun ثوره في هذا البحث عندما قدم خوارزميه LKH. وقد اضاف محرك رياضي قوي لتوجيه محرك البحث المحلي: The Held-Karp Lower Bound. وذلك بدمج Subgradient Optimaization مع محرك تبديل الوصلات LK swapping engine, بذلك اصبحت خوارزميه LKH اقوى خوارزميه حل لمشكله TSP على الاطلاق.

في عام 2017, اصدر العالم Helsgaun خوارزميه LKH-3, حيث طور عده اصدارات من المشكله, مثل اضافته خاصيه توجيه عده بائعين, نوافذ الوقت, والتكديس, مما جعلها تتراس على عرش الخوارزميات الاكفئ في حل المشكله عند ازدياد حجمها.

اسطاعه خوارزميه LKH-3 تحقيق سرعتها بتقسيم المشكله الى اجزاء اصغر مركزه وذات كفاءه عاليه.

2.2 الخطوة الاولى: تهيئه البيانات

في المرحله الاولى نعد بيانات المدن ونقوم بحساب الوصلات بينها ونخزنها في Arrays تسهل التعامل معها.

3.2 الخطوة الثانيه: The Held-Karp optimaization loop

1.3.2 القاعده الرياضيه: Lagrangian Relaxation

قبل تبديل اي وصله, تقوم LKH بتشغيل subgradient optimaization لحساب معاملات العقوبه او التحول للمسافه (penalized distances) لكل مدينه. وثم تنشئ بنيه بيانيه تدعى 1-Tree في كل دوره. اذا كانت مدينه معينه مزاره الكثير من المرات ($degree > 2$), قيمه الpenalty خاصتها ستزداد, مما يجعل التوصيل معها مكلف اكثر, اما اذا كانت معزوله ($degree < 2$), تنقص قيمه الpenalty خاصتها, بعد مئات من هذه الدورات, تؤدي هذه العمليه الى تسطح الرسم البياني, مما يرفع الحد الادنى حتى يصل الى ما دون الحل المثالي للمشكله مباشرةً.

يمكن تمثيل مشكله البائع المتجول برقم وحيد: وهو تقليل اجمالي تكلفه الرحله مع مراعاة القيد الذي ينص على ان كل مدينه يجب ان يكون لها وصلتين فقط ($degree = 2$) بالضبط دون انشاء حلقات فرعيه منفصله. ولكن بسبب اجبار الدرجات ان تكون 2 بالضبط لكل المدن في وقت واحد هو ما يجعل المشكله من تصنيف (NP-Hard). استخدم طريقه Held-Karp تكتيكاً يدعى (Lagrangian Relaxation). وبدلاً من اجبار قيد الدرجة بالقوه, نقوم بازاله القيود الثابته وننقلها الى داله تحسين الهدف (penalty). لاي مصوفه من الارقام الحقيقه $\pi = (\pi_1, \pi_2, \dots, \pi_n)$, نعرف مصوفه من قيم π المعدله C' :

$$C'_{ij} = C_{ij} + \pi_i + \pi_j$$

حيث C_{ij} هي المسافة الجغرافية الحقيقية بين المدينة i والمدينة j . والقيم π_j و π_i هي مضاعفات لاغرانج. اذا جمعنا هذه القيم C'_{ij} لكل مدينة، تصبح التكلفة المعدلة لمجموعه مدن تحتوي N وصلات متغيره او تابعه بشكل منهجي تبعاً للدرجات الخاصه بكل مدينة d_{ij} :

$$\begin{aligned} \text{Penalized Cost} = W(\pi) &= \sum_{e \in E'} C'_{i,j} = \sum_{(i,j) \in E'} C_{i,j} + \sum_{i=1}^N d_i \pi_i \\ &= \sum_{e \in E'} C'_{i,j} = \sum_{(i,j) \in E'} (C_{i,j} + \pi_i + \pi_j) \end{aligned}$$

حيث E' هي مجموعه جزئية من مجموعه الوصلات E .

2.3.2 الیه عمل الحلقة: Subgradient Ascent

تبدأ عملية التحسين باستخدام Subgradient ليجاد اقصى قيمه للحد الادنى، تعمل الحلقة على اربع خطوات متكرره:

1. حساب Minimum 1-Tree: باستخدام قيم الاوزان المعدله (اوزان العقوبه) C'_{ij} الحاليه، يتم تشغيل خوارزميه Kuskal عليها ليجاد شجره MST على $N - 1$ مدن، ثم نقوم بوصل المدينة المميزه (المعزوله) معها باستخدام اقصر وصلتين عندها (من حيث قيمه π_i ايضاً). بذلك يكون اجمالي الوزن المعدل الناتج معطى ب $W(\pi)$.

• السبب وراء اختيار البنيه 1-Tree: يعد إيجاد مسار صحيح لمسألة البائع المتجول TSP أمراً بالغ الصعوبة بسبب قيد صارم: يجب أن يكون حلقة واحدة متصلة حيث تتصل كل مدينة بمدينتين أخريين فقط. ويُعتبر إيجاد بنية ذات وزن أدنى وبهذه الخاصية تحدياً كابوساً من فئة NP-Hard.

تتجاوز شجرة 1-Tree هذه الصعوبة بتخفيف القواعد بما يكفي لتسهيل العمليات الحسابية، مما ينشئ بنية تعد بديلاً مثاليًا لمسار TSP. رياضياً، شجرة 1-Tree هي بنية بيانية مبنية على N مدينة تحتوي على N وصله بالضبط وحلقة مغلقة واحدة فقط، ويتم ذلك باستخدام حيلة تفكيك بسيطة من خطوتين:

(أ) أنشئ الشجرة الأساسية، واختر مدينة واحدة لتكون "المدينة الخاصة" (مثل المدينة 0). أزلها تمامًا من الخريطة، ثم أنشئ شجرة امتداد دنيا قياسية (MST) عبر المدن المتبقية $N-1$ باستخدام خوارزمية Kruskal.

(ب) بمجرد العثور على الشجرة الممتدة الدنيا، نعود إلى المدينة 0 ونقوم بتوصيلها قسرًا بالشجرة الممتدة الدنيا الناتجة باستخدام أرخص حافتين متاحيتين لها. بناءً على تكاليفك الحالية.

لأن الشجرة الممتدة الدنيا القياسية لا تحتوي على حلقات، وعقدة إضافية واحدة (المدينة 0) ستضمن إنشاء دورة واحدة بالضبط.

• لماذا تعد 1-Tree مرآة مثالية لمشكلة TSP؟
لنلق نظرة على المتطلبات الهيكلية لمسار صالح في مسألة البائع المتجول TSP مقارنة بـ 1-Tree:

- تحتوي جولة TSP الصحيحة على N من الحواف، متصلة بالكامل، ولها دورة واحدة فقط، ولكل عقدة درجة تساوي 2 بالضبط.

- تحتوي الشجرة أحادية البعد على N حافة متصلة بالكامل، ولها دورة واحدة فقط، لكن درجات عقدها يمكن أن تكون أي قيمة (درجة 1 أو أكثر)، وهو عدد الحواف المتصلة بها.

وهذا يعني أن مسار TSP الصالح هو ببساطة نوع خاص من بنية 1-Tree، حيث تكون درجة كل مدينة مساوية لـ 2، كما أن قواعده وبنيته البسيطة تجعل عملية بنائه سهلة للغاية.

• عملية استخراج الشجرة الممتدة الدنيا MST:
لاستخراج الشجرة الممتدة الدنيا (Minimum Spanning Tree (MST)، نستخدم خوارزمية Kruskal، بما أننا نبني شجرة أحادية، فإننا نتجاهل مدينة واحدة، مما يترك لنا عدد $N - 1$ مدينة، وفي نظرية الرسوم البيانية (Graph Theory)، تتطلب الشجرة الممتدة الدنيا التي تربط مجموعة من الرؤوس V دائمًا $V - 1$ حافة بالضبط لربطها جميعًا دون تكوين حلقة مغلقة، وبالتالي سيكون إجمالي عدد الحواف $(N - 1) - 1$ (المدينة المميزة التي عُزلت).

• الاداء الاساسيه: The Disjoint Set Union (DSU):
أثناء قيام خوارزمية Kruskal بفحص الوصلات المرتبة (من الأرخص إلى الأعلى)، تطرح سؤالاً بسيطاً: "إذا أضفت هذه الوصلة، فهل سننشئ حلقة مغلقة غير قانونية؟"، يجيب هيكل المجموعة المنفصلة (Dis-joint Set) على هذا السؤال فوراً من خلال تتبع المدن المتصلة بالفعل بفروع هيكلية. ولذلك يستخدم مصفوفتين بسيطتين:

- **parents**: يتتبع الجذر المباشر (الأصل) لكل مدينة في مجموعتها، إذا كانت المدينة هي أصلها، فهي جذر مجموعتها المستقلة.

- **rank**: يتتبع عمق فروع الشجرة لمجموعة ما لضمان أنه عند دمج مجموعتين، يتم توصيل المجموعة الأقصر أسفل المجموعة الأطول.

عند بدء تشغيل الخوارزمية، تكون كل مدينة كياناً منفرداً ومعزولاً تماماً؛ إذ لا تعرف أي مدينةٍ منها المدن الأخرى. وتشكل كل مدينة "دائرة اجتماعية" مستقلة خاصة بها، تتألف من عنصر واحد فقط. وتتولى بنية بيانات "المجموعات المنفصلة" Disjoint Set إدارة هذه الدوائر باستخدام عمليتين أساسيتين:

- **Find**: يسأل مدينة: "من هو القائد المطلق (الأصل الجذري) لدائرتك الاجتماعية؟"

- **Union**: يجمع بين دائرتين اجتماعيتين منفصلتين تماماً تحت قيادة واحدة موحدة.

أثناء تكرار الخوارزمية على الحواف المرتبة، تأخذ معرفي مدينتين وتتحقق من جذورهما (كل منهما في دائرتها الاجتماعية أو مجموعتها الخاصة). إذا لم تكن جذورهما متساوية، فإنها تأخذ الحافة بينهما بأمان، ثم تدمج المجموعة ذات الترتيب الأصغر في المجموعة ذات الترتيب الأعلى، بحيث يكون جذر المجموعة ذات الترتيب الأعلى هو جذر المجموعة ذات الترتيب الأصغر. أما إذا كانت الجذور متساوية، فإنها تتخطى كليهما لأن لهما مساراً مشتركاً أقصر، وبالتالي لا يحتاجان إلى حافة مباشرة بينهما من شأنها أن تسبب حلقة مغلقة.

2. حساب Subgradient Vector (G): يمثل التدرج الفرعي الاتجاه الرياضي

لا كبر خطأ، لكل مدينة i ، نحسب كم تنحرف درجتها الحالية d_i في شجرة 1-Tree عن الهدف المراد في أي رحله TSP فعلية (حيث كل مدينة درجتها تساوي 2 تماماً). يمكننا تعريف الخطأ الهيكلي الفردي لأي عقدة i على النحو التالي:

$$G_i = d_i - 2$$

لفهم كيف يُمثل التدرج الفرعي الاتجاه الرياضي لأكبر خطأ، من المفيد الابتعاد عن البرمجة والنظر إلى هندسة التحسين. إذا كنت تتسلق جبلاً في ضباب كثيف وتريد الوصول إلى القمة بأسرع وقت ممكن، فإنك تنظر إلى الأرض تحت قدميك وتخطو في الاتجاه الذي يصعد فيه المنحدر بأكبر انحدار ممكن. في علم التفاضل والتكامل، يُسمى اتجاه الصعود الأكبر بالتدرج (Gradient - grad). ومع ذلك، فإن رسم دالة الهدف في تحسين Held-Karp ليس جبلاً أملساً ومستديراً تماماً. لأن الشجرة أحادية البعد تتكون من حواف حادة منفصلة، يبدو رسم الدالة وكأنه ماسة متعددة الأوجه أو مثل الهرم. يحتوي على حواف وزوايا حادة حيث يفشل حساب التفاضل والتكامل القياسي لأنه لا يمكنك حساب مشتقة تقليدية عند نقطة حادة. هنا يأتي دور التدرج الفرعي (Subgradient). حيث إنه تعميم للتدرج للدوال غير الملساء والمتعرجة.

في المرحلة الثانية، يتمثل هدفنا الأول هو جوله صالحه (Valid TSP Tour). في الجولة الصالحة، يكون لكل مدينة هدف بان تكون درجتها تساوي 2 بالضبط. إذا أنتجت الخوارزمية 1-Tree حيث تكون درجة إحدى المدن 4، فإننا نواجه خطأً هيكلياً. أي ان المدينة مكتظة. إذا كانت درجة إحدى المدن تساوي 1، فإننا نواجه خطأً أيضاً؛ المدينة معزولة.

إذا كانت المدينة مزدحمه ($d_i = 4$)، يكون تدرجها موجباً ($4 - 2 = +2$). إذا كانت المدينة منعزله او طريق مغلق ($d_i = 1$)، يكون تدرجها سالباً ($1 - 2 = -1$). وإذا كانت المدينة في حاله دوره TSP مثاليه ($d_i = 2$)، ينهار تدرجها نحو الصفر ($2 - 2 = 0$).

لنتذكر صيغة إجمالي "التكلفة العقابية" المعممة الخاصة بنا:

$$W(\pi) = \sum_{(i,j) \in E'} C_{i,j} + \sum_{i=1}^N d_i \pi_i$$

لننظر كيف يؤثر تغيير عقوبة مدينة واحدة π_i على إجمالي التكلفة المعاقبة $W(\pi)$. إذا اعتبرنا حواف شجرة 1-Tree المختارة ثابتة مؤقتًا، فيمكننا حساب المشتقة الجزئية للتكلفة بالنسبة لعقوبة تلك المدينة:

$$\frac{\partial W}{\partial \pi_i} = d_i$$

يُشير هذا إلى أن معدل التغير الحالي (الميل) لدالة التكلفة على طول المحور π_i يساوي تماماً الدرجة الحالية للعقدة، d_i . ومع ذلك، فإننا نسعى لتعظيم الحد الأدنى، ولكن يجب علينا القيام بذلك دون السماح "للعقوبات" (penalties) بتشويه المخطط البياني إلى ما لا نهاية. وتقتضي مسألة التحسين المزدوج (dual optimization problem) تحقيق توازن في النظام، حيث تقوم خطوة التعديل المستهدفة بضبط الميل بالنسبة للقيد المفروض (أي الدرجة المستهدفة البالغة 2)؛ وعند طرح القيد المستهدف، يصبح متجه الميول الاتجاهية كما يلي:

$$\nabla W(\pi) \approx \begin{bmatrix} d_1 - 2 \\ d_2 - 2 \\ \vdots \\ d_N - 2 \end{bmatrix} = \begin{bmatrix} G_1 \\ G_2 \\ \vdots \\ G_N \end{bmatrix} = \vec{G}$$

هذا المتجه $\vec{G}$ هو التدرج الفرعي الخاص بنا.

يخبرنا Subgradient بالاتجاه الذي يجب سلكه لتعديل أخطائنا البنيوية وماهي النقاط التي علينا تعديلها. على سبيل المثال، إذا كانت المدينة

ذات درجة تساوي 5 ($G_i = +3$). يكون الميل ايجابى واتجاه subgradient vector يُؤشر بشده نحو الاتجاه الموجب. ومع التقدم في الحلقات, يسبب ذلك ازدياد قيمه π_i لتلك المدينه بشكل اقوى. وهذا يجعل الوصلات المتصله بتلك المدينه اكبر بالوزن واغلى مع كل حلقه, مما يجبر خوارزميه Kruskal على اقتطاع الوصلات عاليه التكلفه منها.

وعلى العكس تماماً, اذا كانت المدينه ذات درجه منخفضه مثل 1 ($G_i = -1$). في هذه الحاله يُؤشر متجه subgradient نحو الاتجاه السالب. ومع الوقت, ستخفض قيمه π_i لتلك المدينه. مما يجعل الوصلات المتصله معها اخص في التكلفه, ويجبر ذلك خوارزميه Kruskal على اضافته المزيد من الوصلات لتلك المدينه.

3. حساب حجم الخطوه (Step Size (t): لايمكننا ببساطه تغيير قيم π بشكل اعمى؛ فعل ذلك سيؤدي انحراف النظام بشكل كبير وعشوائي عن القيمه الفضلى. يتم حساب حجم التغير عن طريق استخدام متغير setp-size scalar t, مقدار By Held and Karp's classic convergence formula :

$$t = \lambda \frac{\text{Target Bound} - W(\pi)}{\sum_{i=1}^N (G_i)^2}$$

حيث:

- λ Lmabda: هو معامل التخمين التكيفي والذي يبدأ حول عدد حقيقي ويتلاشى نحو الصفر.
- Target Bound: هو تقدير للقيمه الحقيقيه للسقف (Upper Bound), غالباً ما يحسب ابتداءً باستخدام خوارزميه بحث سريعه مثل nearest neighbor, او تعدل بواسطه الحلقه الخارجيه.
- $\sum (G_i)^2$: مجموع الاخطاء للتربيع, ذلك كان الشكل العام لتوزيع المدن عشوائياً والدرجات بعيدة عن 2, تكبر قيمه المقام, مما يجعل حجم الخطوه اصغر واكثر استقراراً.

4. تحديث قيم π : اخيراً, يتم تعديل مصفوفه قيم العقوبات (او يتم تعديلها لكل مدينه) من اجل الدور التالى: اذا كانت درجه المدينه عاليه, قيمه π خاصتها تزداد. في الدور التالى, كل الوصلات المتصله معها تصبح "اغلى", يدفع ذلك خوارزميه Kruskal على تجنبها. وبالعكس, المدن المنفيه او ذات الوصله الواحده يحصلون على

قيمه تعديل (π) سالبه, مما يجعل وصلاتهم "ارخص" واكثر جذباً للخوارزميه في دوره التاليه.